# Vision–Language Guided Quadruped Navigation with Reinforcement Learning Control

Chen Yuan Wang[1], Hector Lugo[1], Christian I. Narcia-Macias[1], Daniel Masamba[1], Pengfei Gu[1], Haoteng Tang[1], Erik Enriquez[1], Dongchul Kim[1]
{[1]Department of Computer Science}, University of Texas Rio Grande Valley
{chenyuan.wang01, hector.lugo02, christian.narcia01, daniel.masamba01, pengfei.gu01, haoteng.tang, erik.enriquez01 , dongchul.kim}@utrgv.edu

***Abstract*—Reinforcement learning (RL) has produced robust low-level controllers for quadruped locomotion, while vision–language models (VLMs) have demonstrated strong semantic reasoning over images and natural language. Most prior work that fuses these modalities for legged robots either trains end-to-end vision–language–action (VLA) models, which require large expert datasets, or relies on additional sensing such as LiDAR or depth cameras. This paper investigates whether a frozen, off-the-shelf VLM can act as a high-level navigator for a separately trained RL controller using only monocular RGB input and proprioceptive state. We propose a hierarchical VLM+RL framework in which the VLM consumes a task prompt and an egocentric image, and emits a heading and forward/stop command that the RL controller follows through low-level joint torques. To improve prediction stability, we introduce a multi-pass inference pipeline that decomposes navigation into scene understanding, steering, and stop sub-prompts, each sampled multiple times. We evaluate the system in a MuJoCo environment containing six common indoor objects across 60 scenarios of varying difficulty. The proposed system achieves up to 100% steering success in unobstructed scenes and 63.3% in hard scenes with obstacles, while a bounding-box-based stopping heuristic outperforms direct VLM stop prediction by 68–78 percentage points across difficulty tiers (78.9–86.4% vs. 8.2–14.4%). Ablations show that multi-pass inference and lower update frequencies remain competitive, and that direct VLM stop prediction and hard obstacle reasoning remain open problems for monocular RGB-only navigation.**

***Index Terms*—Vision–language models, reinforcement learning, quadruped locomotion, robotic navigation, hierarchical control, MuJoCo.**

## I. Introduction

Deep reinforcement learning (RL) has enabled stable and agile locomotion on quadruped and humanoid platforms, using either custom-shaped reward functions over velocity, orientation, and actuator effort [1] or imitation-based rewards that track motion-capture references [2]. These controllers can robustly traverse rough terrain and follow commanded velocities, but they typically operate over a fixed proprioceptive observation space and cannot directly interpret what their environment *means*. Tasks such as "walk to the bookshelf and stop in front of it" require semantic grounding that hand-crafted observations rarely provide.

Large language models (LLMs) and their multimodal extensions, vision–language models (VLMs), offer a complementary capability: given a natural-language instruction and an image, they can identify objects, describe spatial relations, and localize regions through bounding boxes [3]–[5]. This makes them attractive as a high-level planning component for robotic systems. However, two practical obstacles arise. First, VLMs emit discrete tokens at sub-Hz to single-Hz throughput on commodity hardware, while quadruped joint controllers demand 50–100 Hz updates. Second, end-to-end vision–language–action (VLA) models that predict joint actions directly from multimodal input, exemplified by RT-2 [6] and OpenVLA [7], require large amounts of expert demonstration data and significant compute [8].

A natural compromise is a hierarchical decomposition: the VLM provides slow, semantic, navigation-level decisions, and a fast RL policy handles low-level joint control. This idea has a clear ancestor in SayCan [9], which pairs an LLM with learned affordance value functions to ground language plans in manipulation skills. Related embodied foundation-model systems such as PaLM-E [10] directly integrate multimodal sensory inputs with language models, while LM-Nav [11] composes pre-trained language, vision-language, and navigation models for real-world language-conditioned navigation. For navigation specifically, recent zero-shot systems such as ZSON [12], CoW [13], and VLFM [14] show that semantic or vision-language representations can support object-goal navigation, though these systems generally rely on learned navigation policies, depth observations, or occupancy maps. For legged platforms, NaVILA [15] augments a VLM/VLA navigator with LiDAR-derived height maps; LeVERB [16] trains an end-to-end whole-body VLA model with both first- and third-person images; GOAT [17] targets multi-modal goal specification using RGB-D sensing; and Helpful DoggyBot [18] pairs a frozen VLM with an RL controller on a quadruped using a third-person fisheye and an egocentric RGB camera. While these methods demonstrate the value of integrating language with locomotion, they typically rely on additional sensors (LiDAR, depth cameras), additional camera viewpoints, or task-specific fine-tuning of the VLM itself.

In this paper, we ask a narrower but useful question: can a frozen, off-the-shelf VLM, without fine-tuning and using only a single monocular RGB frame, drive a separately trained RL locomotion policy to perform instruction-following navigation? Each constraint is chosen to keep the system cheap to deploy and broadly reusable. Keeping the VLM frozen avoids the expert demonstration datasets and training

1946-0759/26/$31.00 ©2026 IEEE
DOI 10.1109/ICMLA69890.2026.00094

compute required by end-to-end VLA models [6], [7]. It also removes the need for task specific VLM data collection and allows the navigator to benefit from future VLM improvements by simply replacing the model. Restricting perception to a single monocular RGB frame—rather than depth, LiDAR, or additional viewpoints [15], [17], [18]—lowers hardware cost, weight, and power, matches the RGB distribution these models were pretrained on (so a frozen, general-purpose model can be used), and keeps each VLM query lightweight by avoiding the extra visual tokens that depth or multi-view inputs would add on every call. Delegating low-level control to a separately trained RL policy keeps locomotion agnostic to the command source and lets it run at the 50–100 Hz joint control requires, while the VLM is queried only at its native sub-Hz rate. The trade-off for these choices is reduced sensing and no task-specific adaptation; our central question is how much of the navigation problem a general-purpose VLM can solve under them, and which subproblems still require specialized data or sensors. Our contributions are:

- A hierarchical VLM+RL navigation framework in which a frozen VLM, given a task prompt and an egocentric RGB image, emits heading and forward/stop commands that an RL controller follows through low-level joint torques.
- A three-pass, multi-sample VLM inference pipeline that decomposes navigation into scene understanding, steering, and stop sub-prompts, with each sub-prompt sampled three times and aggregated by majority vote (boolean) or median (continuous) to improve consistency.
- An evaluation in a MuJoCo simulation of a Unitree Go2 quadruped [19] across 60 scenarios spanning three difficulty tiers and six common indoor objects (chair, couch, table, bookshelf, cardboard box, trash can), including ablations over prompt design (continuous vs. discrete steering, bounding-box vs. direct stopping, multi-pass vs. single-pass) and VLM update frequency.

## II. Related Work

### A. Reinforcement Learning for Legged Locomotion

Policy-gradient methods such as Trust Region Policy Optimization (TRPO) [20] and Proximal Policy Optimization (PPO) [21] are well-suited to the continuous state and action spaces of legged robots. Hwangbo *et al.* [1] demonstrated that PPO-trained policies for the ANYmal quadruped can transfer from simulation to hardware with minimal fine-tuning, and motion-imitation methods such as DeepMimic [2] have shown that physically-simulated characters can reproduce reference motions through reward shaping. Common to these approaches is reliance on proprioceptive state and pre-specified velocity or heading commands; none provide a mechanism for grounding such commands in visual or linguistic context.

### B. Vision–Language Models for Robot Control

VLMs combine a vision encoder, typically a Vision Transformer [3], with an autoregressive language model based on Transformer self-attention [22]. Recent open-weight VLMs [4], [5] can perform object grounding, relative spatial reasoning, and bounding-box localization from a single prompt. A growing body of work fuses VLMs with robotic control. End-to-end VLA models such as RT-2 [6] and OpenVLA [7] predict joint or end-effector actions directly from multimodal input, but require curated demonstration datasets; PaLM-E [10] similarly shows the value of training embodied multimodal language models across diverse sensorimotor tasks. A complementary direction uses frozen foundation models as zero-shot high-level planners: SayCan [9] combines an LLM with learned affordance value functions for manipulation, LM-Nav [11] composes pre-trained models for language-conditioned navigation, and ZSON [12], CoW [13], and VLFM [14] show that semantic or vision-language representations can support object-goal navigation without task-specific ObjectNav supervision. Navigation-specific foundation models such as ViNT [23] train goal-conditioned visual policies on large robot navigation datasets, whereas our focus is on using a frozen VLM only as a high-level command generator for a separately trained locomotion controller. For legged platforms specifically, NaVILA [15] pairs a VLA navigator with an RL low-level controller using RGB and LiDAR-derived height maps; LeVERB [16] learns hierarchical whole-body humanoid control from latent language instructions using both egocentric and third-person images; GOAT [17] performs lifelong multimodal goal navigation with RGB-D sensing; and Helpful DoggyBot [18] fuses a frozen VLM with an RL controller on a quadruped using a third-person fisheye view alongside egocentric RGB. VLM-Predictive Control [24] uses VLM in-context history and replanning for adaptive obstacle behaviors. Recent empirical work [25] further asks whether pretrained vision–language embeddings alone, without specialized modules, can guide navigation. Compared with these, our system fixes a frozen VLM, uses only a monocular egocentric RGB image, and does not introduce LiDAR, depth, or third-person views. The cost is reduced sensing, and one of our questions is whether a VLM can compensate.

## III. Method

### A. Problem Definition

A quadruped equipped with a forward-facing monocular RGB camera is placed in a room containing one named target object and zero or more distractor objects on the floor (Fig. 1). Given a natural-language task prompt of the form "Walk towards the [target]" and the egocentric RGB image, the system must walk the robot toward the target and stop in front of it without colliding with any non-target object. The evaluation assumes that the target is visible in the initial egocentric frame and that the floor is flat; only forward translation and yaw are required. The navigator has no temporal memory or active search behavior, so if the target is not detected at a later update, it stops rather than attempting recovery. The task assumes the prompt names the target correctly. Position information for the target and distractors is *not* provided to the policy and must be inferred from the image.



Fig. 1. Third-person view of the Unitree Go2 quadruped in the MuJoCo simulation environment. The robot must walk from its starting pose toward a named target object while avoiding distractors using only a forward-facing monocular RGB camera and proprioceptive state.

### B. System Architecture

The architecture is shown in Fig. 2. At every navigator update step, the VLM consumes (i) the task prompt and (ii) the most recent $336\times336\times3$ egocentric RGB frame, and produces a navigator command consisting of a target $x$-velocity, a $y$-velocity (zero in our experiments), and a heading. The RL controller receives the navigator command together with the robot's proprioceptive state—root position and velocity, joint positions and velocities—and outputs joint torque values at the simulator's control frequency. The VLM is invoked at a much lower frequency than the RL controller, which mirrors the practical inference budget of contemporary VLMs.

### C. Reinforcement Learning Controller

The RL controller acts as the low-level locomotion module. The observation space $\mathcal{S} \in \mathbb{R}^{38}$ consists of 35 proprioceptive states (joint positions, velocities, root orientation) concatenated with a 3-dimensional goal command (target $x$-velocity, $y$-velocity, and heading) provided by the VLM. The action space $\mathcal{A} \in \mathbb{R}^{12}$ consists of target joint torques normalized to $[-1, 1]$.

During training in MuJoCo [26] using the Unitree Go2 model [19], target velocities and headings are sampled uniformly: $x$-velocity in $[0, 1.5]$ m/s, $y$-velocity fixed at 0 m/s, and heading in $[-\pi, \pi]$. The reward function $r_t$ is designed to encourage command tracking while ensuring stable locomotion:

$$r_t = w_v \exp(-k_v e_v^2) + w_h \exp(-k_h e_h^2) - \sum_i w_{p,i}\, p_i \quad (1)$$

where $e_v$ and $e_h$ represent the velocity and heading tracking errors, and $p_i$ represents regularization penalties including joint acceleration, actuator torque, action rate changes, and foot symmetry to promote smooth, energy-efficient gaits. Episodes terminate after 1,000 steps or upon detecting a fall (free-joint height $< 0.25$ m).

Because the controller is trained to follow arbitrary velocity/heading commands, it acts as a generic command-following policy and is fully agnostic to the source of those commands. This separation is what allows us to plug in a frozen VLM at inference without any joint training.

### D. Vision–Language Model Navigator

We use Qwen3.5-9B from Hugging Face [5] as the navigator with thinking mode disabled. The VLM weights are not modified.

*a) Multi-pass decomposition.:* A naive design issues a single composite prompt covering target identification, heading estimation, and stop decision. We instead decompose navigation into three sub-prompts, illustrated in Fig. 3:

- **Scene Understanding Prompt.** Extracts the target noun phrase from the task prompt and asks the VLM to confirm whether the target is visible in the egocentric image. If absent, the navigator emits stop command to the robot.
- **Steering Prompt.** Asks the VLM to recommend a heading change toward the target while avoiding any obstacle along the path.
- **Stop Prompt.** Asks the VLM whether the robot should stop at the current step.

Each sub-prompt is sampled three times and aggregated by majority vote (boolean outputs) or median (continuous outputs). This reduces the variance of single-shot VLM responses without requiring fine-tuning.

*b) Steering prompt variants:* We compare a *continuous* variant, in which the VLM outputs a normalized relative heading change in $[-1, 1]$, and a *discrete* variant, in which it selects from {slight left, left, straight, right, slight right}. The continuous form gives finer control but demands more from the VLM; the discrete form constrains the output space.

*c) Stop prompt variants:* We also compare two ways of deciding when to stop: a *bounding box* variant in which the VLM emits a bounding box around the target, and we trigger the stop when the bottom of the box exceeds a threshold $y$-coordinate in the image (a proxy for the target being close); and a *direct* variant in which the VLM emits a boolean stop decision.

*d) Single-pass baseline:* As an ablation, we collapse all three sub-prompts into a single composite prompt requesting target presence, heading, and stop in one VLM call.

### E. VLM Update Frequency

The RL controller runs every simulator step. The VLM, which is far slower, runs every $N$ environment steps, with the most recent navigator command held constant in between. We evaluate $N = 20$ and $N = 50$.

## IV. Experiments

### A. Experimental Setup

The environment is a $15\times15$ m room with grey walls and a checkered floor in MuJoCo. Six common indoor objects can be placed inside: chair, couch, table, bookshelf, cardboard box, and trash can. The robot is the Unitree Go2 quadruped [19].

We construct 60 scenarios partitioned by difficulty:

- **Easy** (18 scenarios): a single target object on a clear floor.
- **Moderate** (12 scenarios): the target plus one or more distractor objects, but no obstacle directly between robot and target.

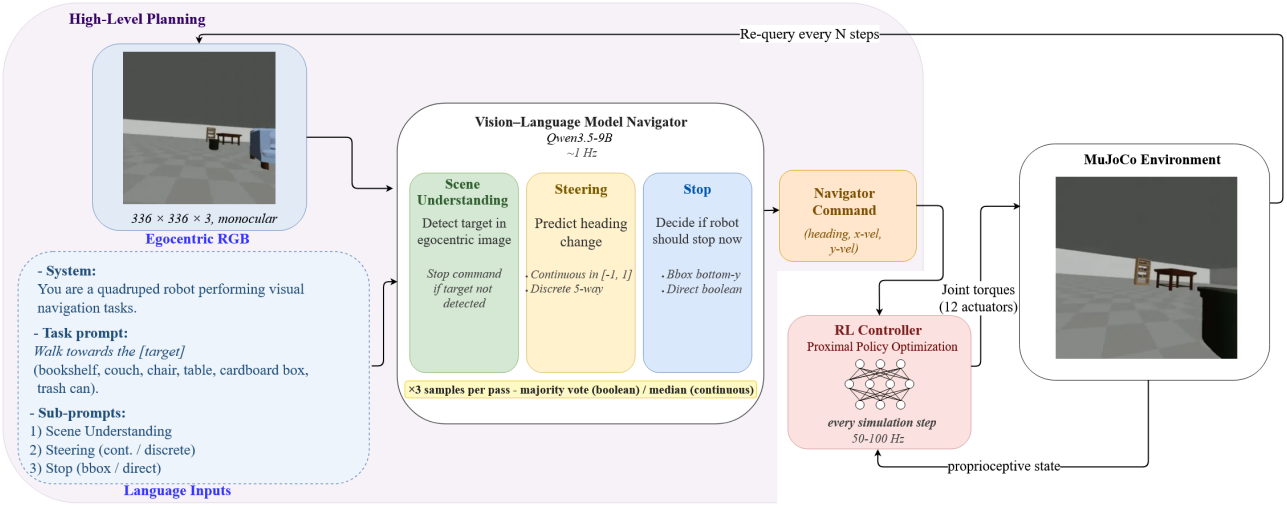



Fig. 2. Overview of the hierarchical VLM+RL navigation framework. At each navigator update (every N=20 or N=50 simulation steps), the frozen Qwen3.5-9B VLM receives a task prompt and a 336×336×3 monocular egocentric RGB frame and issues three sequential sub-prompts: (1) Scene Understanding, which detects whether the target is visible; (2) Steering, which predicts a heading change toward the target (continuous $\in$ [-1,1] or discrete 5-way); and (3) Stop, which decides whether to halt (bounding-box heuristic or direct boolean). Each sub-prompt is sampled three times and aggregated by majority vote (boolean) or median (continuous). The resulting navigator command (heading, x-velocity, y-velocity=0) is passed to a PPO-trained RL controller, which runs at 50–100 Hz and outputs joint torques to 12 actuators in the MuJoCo environment. Proprioceptive state is fed back to the RL controller only.

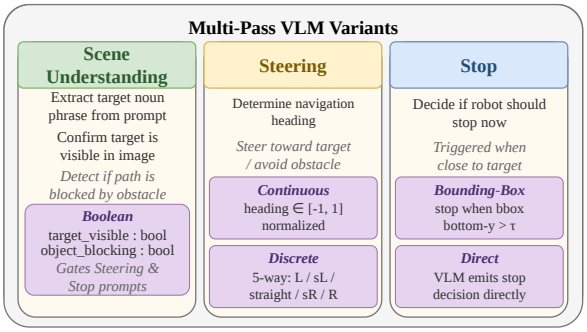



Fig. 3. Multi-pass VLM inference. Three sub-prompts are issued per navigator step (Scene Understanding, Steering, Stop). The Steering prompt has continuous and discrete variants; the Stop prompt has a bounding-box variant (compute target's bottom-$y$ in image and stop when $y > \tau$) and a direct variant (VLM emits stop boolean directly). Each sub-prompt is sampled three times and aggregated by majority vote (boolean) or median (continuous).

- **Hard** (30 scenarios): the target plus distractors, with at least one obstacle on the straight path between robot and target, requiring the policy to deviate around it.

Each VLM configuration is run 10 times per scenario.

### B. Evaluation Criteria

We report two complementary success rates:

- **Steering success**: the robot reaches a position within 3 m of the target without colliding with any non-target object, and stops the episode within that radius.
- **Stopping success**: the robot stops within 2 m of the target without colliding with the target.

TABLE I
VLM NAVIGATOR CONFIGURATIONS EVALUATED

| Configuration | Update interval | Steering | Stop |
|---|---|---|---|
| Continuous Heading $N$=50 | 50 steps | Continuous | Bounding Box |
| Continuous Heading $N$=20 | 20 steps | Continuous | Bounding Box |
| Discrete Heading $N$=50 | 50 steps | Discrete | Bounding Box |
| Discrete Heading $N$=20 | 20 steps | Discrete | Bounding Box |
| Direct Stop $N$=50 | 50 steps | Continuous | Direct |
| Direct Stop $N$=20 | 20 steps | Continuous | Direct |
| Single Prompt $N$=50 | 50 steps | Continuous | Bounding Box |
| Single Prompt $N$=20 | 20 steps | Continuous | Bounding Box |

Although both criteria include a terminal-distance condition, they evaluate different aspects of the task: the 3 m condition in steering success is a coarse goal-region check that confirms the navigator safely routes the robot into the target vicinity, whereas stopping success uses a stricter 2 m threshold and target-collision constraint to evaluate the precision of the final stop.

### C. Experimental Conditions

We evaluate eight configurations summarized in Table I. Configurations differ along three axes: steering prompt variant, stop prompt variant, and update interval $N$. The Continuous Heading and Discrete Heading rows isolate the steering variant; the Direct Stop rows isolate the stop variant; the Single Prompt rows test the multi-pass vs. single-pass decomposition.

Training and evaluation were performed on a compute cluster node with an NVIDIA A100 80 GB GPU and six cores of an Intel Xeon Gold 6330 CPU.

TABLE II
HEADING ESTIMATION ERROR ON THE SYNTHETIC RED-DISK TEST (DEGREES, LOWER IS BETTER). MEAN ABSOLUTE ERROR AVERAGED ACROSS SIX TARGET POSITIONS.

| VLM | MAE (deg) |
|---|---|
| GPT 5.2 [27] | 2.59 |
| Gemini 3 [28] | 2.59 |
| Qwen 3.5-9B [5] | 2.49 |

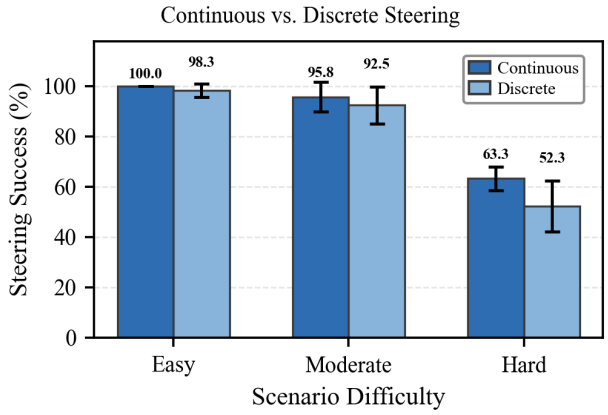



Fig. 4. Steering success rate of the integrated VLM+RL system, comparing continuous and discrete steering prompt variants across difficulty tiers. Error bars denote the variability across the 10 evaluation runs, indicating the run-to-run variation in steering performance.

### D. VLM Heading Feasibility Test

Before evaluating the integrated stack, we conducted a small VLM feasibility test, isolating heading estimation. We rendered single images of a red disk placed at known $(x, y, z)$ positions in front of a synthetic camera and asked three VLMs (GPT 5.2, Gemini 3, and Qwen3.5-9B model used in our integrated stack) to report the heading toward the disk in degrees. This validates whether modern VLMs can produce navigation-grade heading estimates from a monocular RGB frame at all.

## V. RESULTS

### A. Heading Feasibility

Table II shows that all three VLMs estimate heading toward a salient red disk to within roughly 2.5° of ground truth. This indicates that monocular heading estimation is within reach of frozen VLMs, justifying the use of one as a navigator.

### B. Continuous vs. Discrete Steering

Fig. 4 shows that both steering variants are near-perfect on easy scenarios (continuous 100.0%, discrete 98.3%). On moderate scenarios, the gap widens slightly (95.8% vs. 92.5%, a 3.3 percentage-point margin), before widening substantially on hard scenarios, where the continuous variant achieves 63.3% versus 52.3% for the discrete variant—a 11 percentage-point margin. The discrete variant constrains heading to a small set of choices, which limits the navigator's ability to thread between obstacles in hard scenes; the continuous variant retains the granularity needed for fine-grained obstacle avoidance.

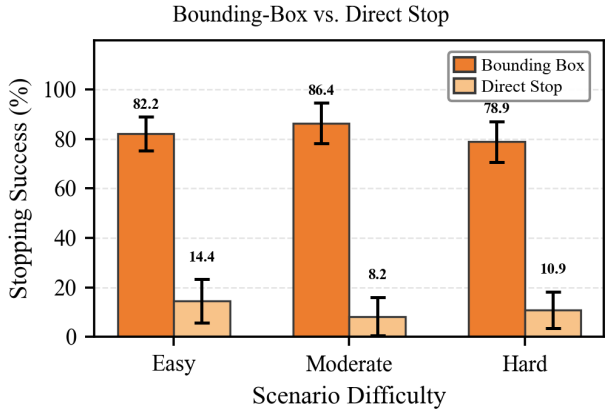



Fig. 5. Stopping success rate (robot halts within 2 m of target without collision) of the integrated VLM+RL system, comparing the bounding-box stop heuristic with direct VLM stop prediction. Error bars denote the variability across the 10 evaluation runs, indicating the run-to-run variation in stopping performance.

### C. Bounding-Box vs. Direct Stopping

Fig. 5 shows a substantial gap: bounding-box stopping reaches 78.9–86.4% across difficulty tiers (easy, moderate, hard), while direct VLM stopping never exceeds 14.4%, a 68–78 percentage-point gap. The VLM struggles to estimate distance from a single monocular image and rarely emits the stop token at the appropriate time, instead either stopping prematurely (mistaking far targets for near ones) or never stopping. The bounding-box heuristic sidesteps absolute depth estimation by using the bottom-edge $y$-coordinate of the predicted box, which correlates monotonically with target proximity for ground-supported objects. The drawback is that the threshold must be tuned per object class.

### D. Multi-Pass vs. Single-Pass Inference

Fig. 6 compares the two prompt architectures across difficulty tiers. Steering success is essentially tied on easy scenarios (both 100%) and stays close on moderate and hard tiers, with multi-pass holding a small steering edge as difficulty increases. Stopping success shows the opposite tendency: single-pass matches or slightly exceeds multi-pass across all tiers, most visibly on moderate scenarios (94.5% vs. 86.4%). A plausible explanation is that a single composite prompt preserves context across subtasks, improving proximity judgment and avoiding the cascading errors a staged pipeline can introduce, whereas the staged decomposition gives finer control for obstacle avoidance. Overall the results suggest a modest steering–stopping trade-off between the two designs.

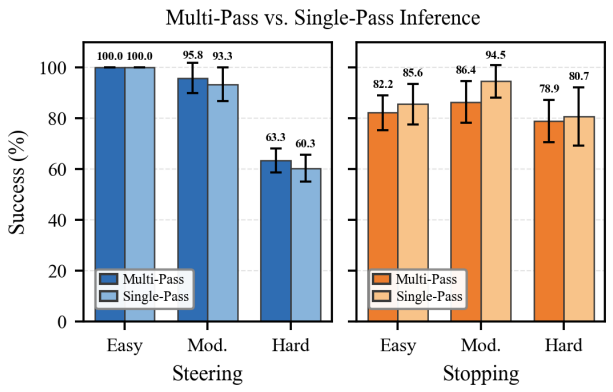



Fig. 6. Multi-pass vs. single-pass inference. Multi-pass slightly favors steering, while single-pass favors stopping success. This difference diminishes in the hard-scenario tier.

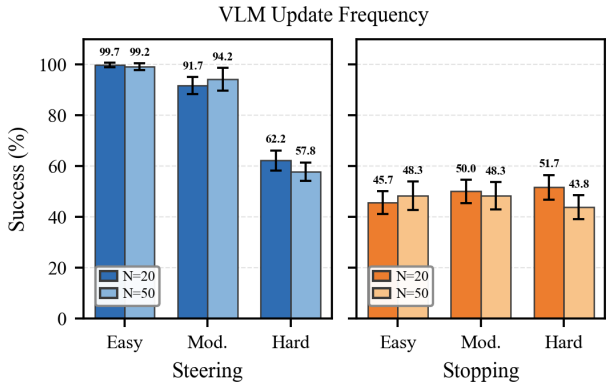



Fig. 7. Effect of VLM update interval on steering and stopping success. Steering performance is close between $N$=20 and $N$=50. Hard-scenario stopping shows a more notable gain at $N$=20 (51.7% vs. 43.8%), reflecting more frequent stop opportunities near the target.

### E. VLM Update Frequency

Fig. 7 compares VLM update intervals of 20 and 50 environment steps. Steering differences are small in absolute terms: hard-scenario steering is 62.2% at $N$=20 and 57.8% at $N$=50. Stopping shows a slightly larger trend favoring $N$=20, with hard-scenario stopping reaching 51.7% at $N$=20 versus 43.8% at $N$=50, consistent with more frequent stop opportunities near the target. Because the RL controller continues executing the most recent VLM command at full simulator frequency, the system is tolerant to slow VLM updates when the navigator command does not need to change rapidly. This is consistent with prior observations that hierarchical decompositions tolerate latency at the upper layer [15], [24]. Stopping success for the frequency comparison is averaged across both the bounding box and direct stop variants, as the frequency ablation applies equally to both stopping strategies. The multi-pass vs. single-pass comparison uses only the bounding box stopping variant, as the single-pass condition was not evaluated with direct stop. Therefore, the stopping successes in the VLM update frequency comparisons are noticeably lower than multi-pass vs. single-pass results due to the low success rate of the direct stop method.

## VI. Discussion

### A. What the Frozen VLM Does Well

The heading feasibility test (Table II) and steering results (Fig. 4) show that a frozen VLM can produce useful heading commands from monocular RGB. In our evaluation, continuous steering achieved the highest observed success, while the bounding-box stopping strategy substantially outperformed direct stop prediction. Together, these results suggest that the VLM is most reliable when its output can be grounded directly in image geometry, such as a relative heading or an object bounding box. This supports using the frozen VLM as a slow semantic and geometric navigator, while the separately trained RL policy executes its commands at the locomotion control rate.

### B. What the Frozen VLM Does Poorly

Hard-scenario steering remained limited, reaching 63.3% at best. Failures were commonly associated with collisions after the VLM recognized the target but did not provide sufficient obstacle clearance. This shows that successful target grounding does not necessarily provide reliable traversability reasoning from a single monocular frame.

Direct VLM stopping was the clearest failure, achieving only 8.2–14.4% success across difficulty tiers, compared with 78.9–86.4% for the bounding-box strategy. This gap indicates that the frozen VLM can localize objects more reliably than it can infer metric distance. However, the bounding-box approach requires an object-class-specific threshold $\tau$, reducing the zero-shot generality of the stopping mechanism.

### C. Limitations and Future Work

Beyond these measured failures, the study has several scope limitations. Evaluation is restricted to 60 visually simple MuJoCo scenarios with flat terrain, simplified object models, and targets visible at initialization. Because the navigator has no temporal memory or active search behavior, it stops rather than attempting recovery if the target is subsequently lost. We evaluate command-holding intervals of $N = 20$ and $N = 50$ simulator steps, but do not measure wall-clock VLM latency or test the effects of communication delay, motion blur, uneven terrain, photorealistic appearance, or hardware deployment. The heading feasibility test is also limited to a salient red disk at six positions, and the study includes no non-VLM navigation baseline. Future work should evaluate more realistic visual conditions and hardware, compare against visual-servoing baselines, and investigate temporal memory, target-recovery policies, and monocular depth or multi-frame geometry for more general stopping and safer obstacle avoidance.

## VII. Conclusion

We presented a hierarchical VLM+RL framework for instruction-following quadruped navigation that uses a frozen, off-the-shelf VLM as the high-level navigator and a separately-trained PPO policy as the low-level controller, using only monocular egocentric RGB and proprioceptive state. The main empirical findings are that (i) frozen VLMs can produce reliable heading commands; (ii) bounding-box stop heuristics dramatically outperform direct VLM stop prediction; and (iii) hard obstacle avoidance with monocular RGB remains an open problem. Current VLMs are useful as high-level navigators when paired with simple geometric heuristics; the remaining bottleneck for sensor-light hierarchical pipelines is metric reasoning rather than semantic understanding. Code and additional results will be available at an anonymized repository linked from the camera-ready version.

## References


[1] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, "Learning agile and dynamic motor skills for legged robots," *Science Robotics*, vol. 4, no. 26, p. eaau5872, Jan. 2019.

[2] X. B. Peng, P. Abbeel, S. Levine, and M. van de Panne, "DeepMimic: Example-guided deep reinforcement learning of physics-based character skills," *ACM Transactions on Graphics*, vol. 37, no. 4, pp. 143:1–143:14, Aug. 2018.

[3] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proceedings of the International Conference on Learning Representations (ICLR)*, Virtual, 2021.

[4] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, New Orleans, LA, USA, 2023.

[5] Qwen Team, "Qwen3.5: Accelerating productivity with native multimodal agents," https://qwen.ai/blog?id=qwen3.5, Feb. 2026, accessed: May 2026.

[6] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, P. Florence, C. Fu, M. G. Arenas, K. Gopalakrishnan, K. Han, K. Hausman, A. Herzog, J. Hsu, B. Ichter, A. Irpan, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, L. Lee, T.-W. E. Lee, S. Levine, Y. Lu, H. Michalewski, I. Mordatch, K. Pertsch, K. Rao, K. Reymann, M. Ryoo, G. Salazar, P. Sanketi, P. Sermanet, J. Singh, A. Singh, R. Soricut, H. Tran, V. Vanhoucke, Q. Vuong, A. Wahid, S. Welker, P. Wohlhart, J. Wu, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yan, A. Zeng, and T. Zhang, "RT-2: Vision-language-action models transfer web knowledge to robotic control," in *Proceedings of the 7th Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 229. Atlanta, GA, USA: PMLR, 2023, pp. 2165–2183.

[7] M. J. Kim, K. Pertsch, S. Karamcheti, T. Mees, A. Balakrishna, S. Urain, E. Naming, P. Florence, G. Mann, A. Teti, J. Shi, C. Finn, P. Abbeel, T. Darrell, J. Bohg, D. Sadigh, and P. Liang, "OpenVLA: An open-source vision-language-action model," 2024.

[8] M. J. Kim, C. Finn, and P. Liang, "Fine-tuning vision-language-action models: Optimizing speed and success," 2025.

[9] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. J. Ruano, K. Jeffrey, S. Jesmonth, N. J. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K.-H. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, K. Reymann, M. Ryoo, G. Salazar, P. Sanketi, P. Sermanet, J. Singh, A. Singh, R. Soricut, H. Tran, V. Vanhoucke, Q. Vuong, A. Wahid, S. Welker, P. Wetzel, F. Yu, H. Zheng, and A. Zeng, "Do as I can, not as I say: Grounding language in robotic affordances," in *Proceedings of the 6th Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 205. Auckland, New Zealand: PMLR, 2022, pp. 287–318.

[10] D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, W. Huang, Y. Chebotar, P. Sermanet, D. Duckworth, S. Levine, V. Vanhoucke, K. Hausman, M. Toussaint, K. Greff, A. Zeng, I. Mordatch, and P. Florence, "PaLM-E: An embodied multimodal language model," in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 8469–8488.

[11] D. Shah, B. Osiński, B. Ichter, and S. Levine, "LM-Nav: Robotic navigation with large pre-trained models of language, vision, and action," in *Proceedings of the 6th Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 205. PMLR, 2023, pp. 492–504.

[12] A. Majumdar, G. Aggarwal, B. S. Devnani, J. Hoffman, and D. Batra, "ZSON: Zero-shot object-goal navigation using multimodal goal embeddings," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.

[13] S. Y. Gadre, M. Wortsman, G. Ilharco, L. Schmidt, and S. Song, "CoWs on pasture: Baselines and benchmarks for language-driven zero-shot object navigation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Vancouver, BC, Canada, 2023, pp. 23 171–23 181.

[14] N. Yokoyama, S. Ha, D. Batra, J. Wang, and B. Bucher, "VLFM: Vision-language frontier maps for zero-shot semantic navigation," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, Yokohama, Japan, 2024, pp. 42–48.

[15] A.-C. Cheng, K. Schmeckpeper, D. Aghi, M. Schmittle, S.-F. Chung, A. Swann, C. Zhu, S. Velastin, V. Kumar, M. Bhatt, and S. Gupta, "NaVILA: Legged robot vision-language-action model for navigation," 2024.

[16] H. Xue, A. Abboud, E. Ferrer, D. Ordonez-Apraez, H. Ferrolho, K. Caluwaerts, J. Merel, N. Leonard, D. Rao, and J. Pajarinen, "LeVERB: Humanoid whole-body control with latent vision-language instruction," 2025.

[17] M. Chang, T. Gervet, M. Khanna, S. Yenamandra, D. Shah, S. Y. Min, K. Yang, D. Batra, A. Gupta, C. Paxton, D. S. Chaplot, J. Malik, R. Liu, and O. Maksymets, "GOAT: Go to any thing," 2023.

[18] Q. Wu, Z. Xu, X. Liao, and X. Cheng, "Helpful DoggyBot: Open-world object fetching using legged robots and vision-language models," 2024.

[19] Unitree Robotics, "Unitree Go2 quadruped robot," https://www.unitree.com/go2, 2023, accessed: 2026.

[20] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, and P. Abbeel, "Trust region policy optimization," in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. Lille, France: PMLR, 2015, pp. 1889–1897.

[21] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017.

[22] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, Long Beach, CA, USA, 2017.

[23] D. Shah, A. Sridhar, N. Dashora, K. Stachowicz, K. Black, N. Hirose, and S. Levine, "ViNT: A foundation model for visual navigation," in *Proceedings of the 7th Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 229. PMLR, 2023, pp. 711–733.

[24] A. Sridhar, J. Wu, A. Bohg, and A. Boularias, "VLM-PC: Online adaptive behavior anticipation via vision-language model predictive control," 2024, arXiv:2407.02666.

[25] N. Subedi, A. Haroon, S. Ganguly, S. T. K. Tetteh, P. Koirala, C. Fleming, and S. Sarkar, "Can pretrained vision-language embeddings alone guide robot navigation?" 2025.

[26] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Vilamoura, Portugal, 2012, pp. 5026–5033.

[27] OpenAI, "Introducing GPT-5.2," https://openai.com/index/introducing-gpt-5-2/, Dec. 2025, accessed: May 2026.

[28] Google DeepMind, "Gemini 3: Introducing the latest Gemini AI model from Google," https://blog.google/products/gemini/gemini-3/, Nov. 2025, accessed: May 2026.